EN2 · STARLING AUTOMATION

Living Textbook

A narration-only workflow that turns a lesson script into an animated, interactive roleplay: Codex images and ElevenLabs narration, the Starling storyboard method, EN2 Ultra animation, static cover and option images, FFmpeg assembly and a delivery ZIP.

Workflow explanation · 3 November 2026

1. Intake → 2. Script → 3. Narration audio → 4. Visual plan → 5. Assets → 6. Storyboards → 7. Static images → 8. Animation (EN2 Ultra) → 9. Review → 10. Edit + stitch → 11. ZIP

1. The flow at a glance

#	Stage	Who / what	Output
1	**Intake**	Ask: script (Google Doc or Starling link), style reference image, narrator voice	`intake.json`
2	**Script**	Narration lines, clip split (<= ~11 s of narration per clip), choice mapping for Predict-Observe-Explain, static image list, cast from the script only	`script.json`
3	**Narration audio**	Codex -> ElevenLabs `eleven_v3`; `harvest_audio.py` downloads and measures	`audio/*.mp3`, `audio/narration.json`
4	**Visual plan**	One timing table: per clip a 1x2/1x3 board; per shot required interval (measured), 1-2 s buffer, generated/action/hold ranges, camera, action, visible inventory	`plan.json`
5	**Assets**	Codex images: scene masters, prop / macro / cutaway sheets (characters only if the script has them)	`assets/`
6	**Storyboards**	Codex draws black-and-white panels; `assemble_boards.py` stacks them and prints the captions from plan.json	`storyboards/`
7	**Static images**	Codex: cover page, answer / reason / confidence / check images, no text	`static/`
8	**Animation**	Six-section EN2 prompts from the inspected boards; `en2_upload.mjs` (hash-checked); `video_generate` Ultra; `en2_collect.mjs`	`prompts/`, `renders/`
9	**Review**	`review_cuts.py` (cuts, drift, frame sheets) + visual check per shot; re-time first; one repair per clip	`renders/cuts.json`, `review.json`
10	**Edit and stitch**	`build_edit.py` (spans from observed cuts, optional cover opener) and `assemble_edit.py`	`edit.json`, `output/`
11	**Package**	`package.py`: roleplay.json + ZIP + SHA-256 manifest	`<slug>-living-textbook-<date>.zip`

2. What changed from Starling

Starling	Living Textbook
Dialogue scenes with lip-synced speech	Removed - narration only
Narration-to-dialogue handoff panels	Removed
Character voices per speaker	One narrator voice
Native ChatGPT images in the session	Codex native images through `codex exec` (`codex_run.py`)
ElevenLabs in the session	ElevenLabs through Codex; `harvest_audio.py` downloads
References via Google Drive URLs	Direct EN2 upload through the EN2 MCP (`en2_upload.mjs`), verified by content hash
Captions drawn on the board image	Captions printed in code from plan.json (exact text)
Planned cut times trusted	Observed cuts measured; spans re-timed before any repair
-	Static images: cover, answer, reason, confidence, check (`roleplay.json`)

Starling	Living Textbook
-	Science shot types: macro, cutaway, cross-section, process close-up, experiment close-up

3. Rules that never bend

1. The script decides the cast. A style reference is style only.
2. Audio first; one timing table.
3. Inspect every generated image, board and render; record what was inspected.
4. Ultra renders; each clip submitted once; every job ID recorded; rejected media kept.
5. Re-time before repair; one repair render per clip; then the user decides.
6. Narration is never stretched; footage is trimmed to it.

4. Walkthrough: the stomata pilot

Intake: style reference (style only)

The style reference supplies the faceted low-poly look. It never supplies a character: this lesson has no cast, so nothing in the production shows people.

Style used: faceted low-poly 3D with hard-edged planes, painterly warm shading on each facet, saturated colours and bold flat backgrounds. The reference image itself is not reproduced in this public document.

Narration audio first

ElevenLabs eleven_v3 (Ritika) through Codex; harvest_audio.py downloaded the files and measured the sentence pauses that set every shot boundary: N1 7.92 s, N2 13.68 s (pauses at 4.28, 9.52, 11.23 s).

Assets from Codex



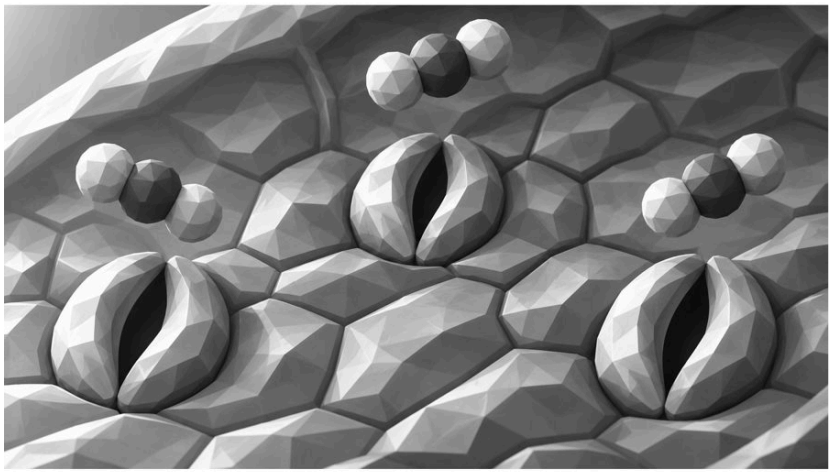
Scene master and science sheets (macro, cutaway, cross-section, experiment). Particle designs stay fixed across every shot.

Storyboards: panels by Codex, captions by code

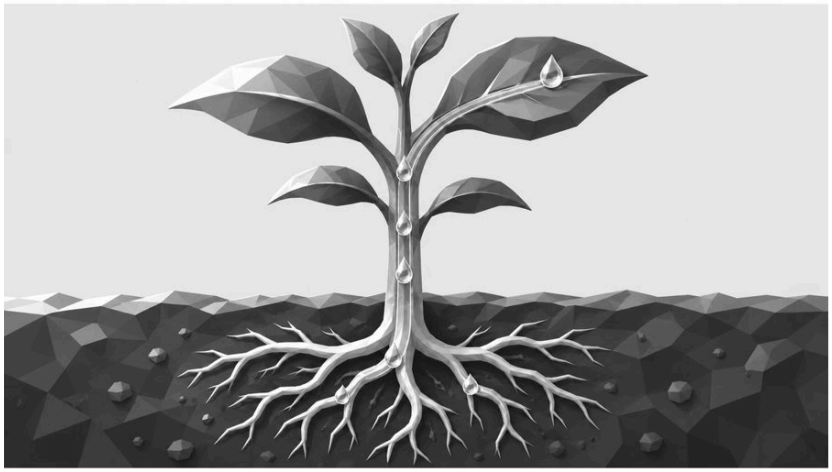
Codex draws each black-and-white panel; assemble_boards.py stacks them and prints the SHOT / time / CAMERA / ACTION strip from plan.json so the text is exact and matches the prompt.



SHOT 1 0-4 s (action 0-3 s, hold 3-4 s)
CAMERA: slow push-in towards the seedling leaf
ACTION: a light breeze tips one seedling leaf up so its pale underside faces the camera



SHOT 2 4-8 s (action 4-7 s, hold 7-8 s)
CAMERA: static extreme close-up, very slow push-in
ACTION: the stomata pores open slightly and CO2 molecules drift down into them



SHOT 3 8-11 s (action 8-10 s, hold 10-11 s)
CAMERA: slow tilt up from roots to leaf
ACTION: blue water droplets travel up from the roots through the stem into the leaf

C1 stomata, 1x3 board: leaf tips up → stomata take in CO2 → water rises from the roots.

Static roleplay images

The cover and one text-free picture per option; the player overlays the option text. Right and wrong options get equal visual weight.

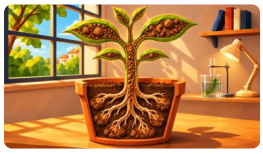


Cover page.

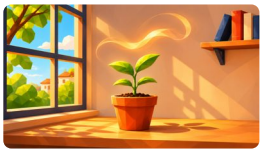





Answer options A-D.







Reason options r1-r5.

EN2 Ultra render and review



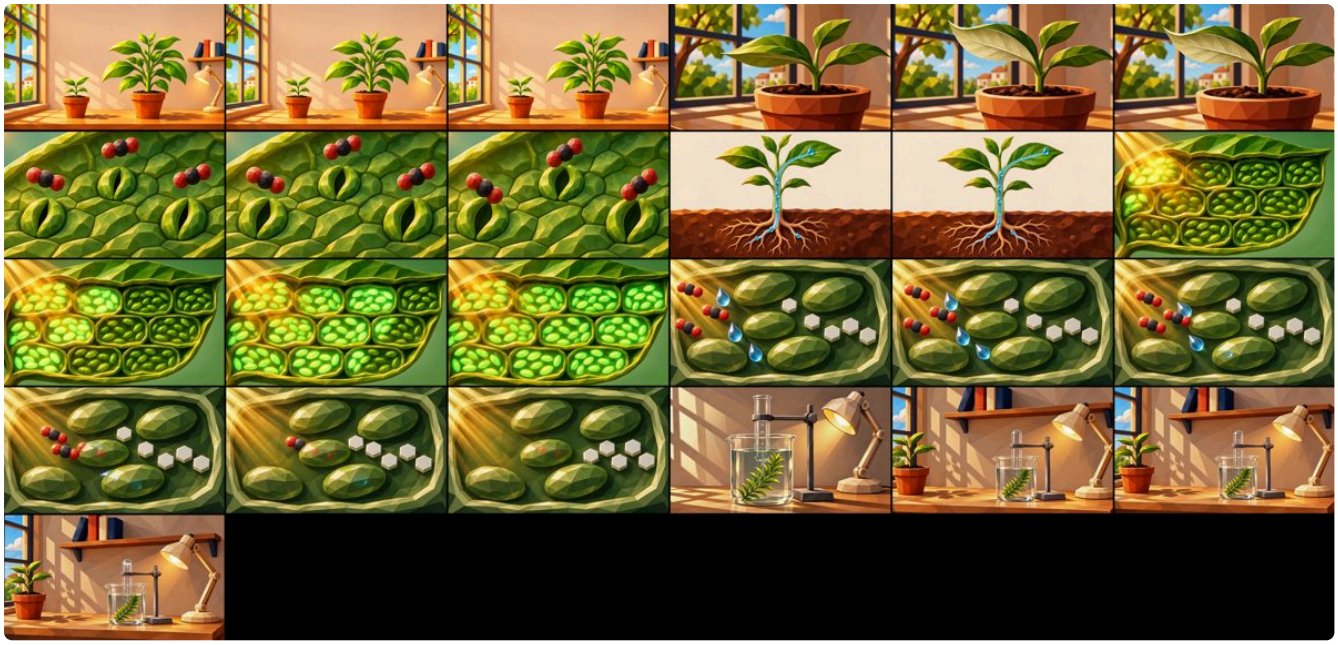
C1 render at 2 frames per second: three shots, hard cuts at 3.71 s and 8.29 s.

Clip	Planned cuts (s)	Observed cuts (s)	Verdict
C1-stomata	[4, 8]	[3.708, 8.292]	verified
C2-chlorophyll	[6]	[3.75]	verified_with_retime
C3-oxygen	[3]	[3.042]	verified

> EN2 placed the C2 cut 2.25 s early. build_edit.py rebuilt the spans from the observed cut and carried the shortfall into the next shot, so the clip was kept without a repair render. Re-time first, repair second.

Edit, stitch, package

assemble_edit.py trims each shot's buffer, joins the shots with hard cuts, discards EN2 sound and lays the original narration underneath; the optional cover opens the video. package.py writes roleplay.json and the ZIP with a SHA-256 manifest.



Final stitched video (cover + three clips), one frame per second.

5. Pilot evidence

- Codex produced 6 colour assets and 9 black-and-white panels in the style reference's faceted low-poly look.
- EN2 Ultra followed the boards: correct layouts, actions, colour from monochrome boards, clean hard cuts, no leaked captions.
- Observed cuts: C1 3.71 / 8.29 (planned 4 / 8), C2 3.75 (planned 6), C3 3.04 (planned 3). C2 was re-timed, not repaired.
- Mistake caught and fixed: a character was invented from the style reference; both affected clips were rejected and redone without people. The rule is now in every skill.
- ElevenLabs via Codex generated the narration (7.92 s, 13.68 s); Codex's sandbox could not download it, so `harvest_audio.py` was added.

Skill: living-textbook-master

Coordinate a Living Textbook production end to end - narration-only educational roleplays built with the Starling storyboard method, Codex images and ElevenLabs audio, EN2 Ultra animation, static cover/option images, FFmpeg stitching and a delivery ZIP.

Living Textbook - Production Master

Living Textbook turns a finished lesson script (a Google Doc or a Starling roleplay link) into a narrated, animated interactive roleplay. It is the Starling workflow with three changes:

1. **No dialogue.** Every clip is narration over picture. There are no speaking characters, no lip-sync, no clean dialogue scenes and no narration-to-dialogue handoffs.
2. **Static roleplay images.** Besides the animated clips, the production makes the cover page and one text-free image for every answer, reason, confidence and check option, because the player shows these as stills.
3. **Science shot types.** Macro inserts, cutaways, cross-sections and experiment close-ups are first-class storyboard shots (see lt-visual-plan).

Everything else follows the Starling storyboard method: audio first, scene masters, labelled black-and-white 1x2/1x3 storyboards, six-section EN2 reference prompts, Ultra renders, full review, one repair, FFmpeg assembly.

Stages and owners

#	Stage	Skill	Exit evidence
1	Intake	lt-intake	intake.json with source, style reference, narrator voice, answered by the user
2	Script	lt-script	script.json: every narration line, clip split, choice mapping and static-image list, matching the source word for word
3	Narration audio	lt-audio	audio/*.mp3 from ElevenLabs (via Codex) + audio/narration.json with measured durations and pauses
4	Visual plan	lt-visual-plan	plan.json timing table: clips, shots, required/generated ranges, camera, action, visible inventory
5	Assets	lt-images	scene masters and prop/macro sheets, inspected
6	Storyboards	lt-images	panels + assembled labelled boards (storyboards/*.png, boards.json), inspected
7	Static images	lt-static-images	static/cover.png and option images, inspected
8	Animation	lt-animation	prompts/*.txt, uploads.json (hash-matched), renders/jobs.json with real job IDs
9	Review	lt-review	renders/cuts.json + review.json with a verdict for every clip
10	Assembly	lt-assembly	edit.json, output/final.mp4, output/segments/, delivery ZIP

Finish and inspect each stage before starting the next. A correction updates its own stage and invalidates only what depends on it.

Providers (fixed)

Need	Provider	How
Images (assets, panels, cover, options)	Codex native image generation	`pipeline/codex_run.py` (`codex exec` with the style reference and asset refs attached)
Narration	ElevenLabs via Codex's connected ElevenLabs app, model eleven_v3	`codex_run.py` then `pipeline/harvest_audio.py` (Codex's sandbox cannot download; the harvester does)
Animation	EN2 reference-to-video, **Ultra only** (`quality=true`)	prompts by hand from inspected boards; `pipeline/en2_upload.mjs` , `video_generate` , `pipeline/en2_collect.mjs`
Edit	Local FFmpeg	`review_cuts.py` , `build_edit.py` , `assemble_edit.py` , `package.py`

Do not silently substitute providers. Never build animation in code (three.js / anime.js); the storyboard method covers science content.

Hard rules

- **The script decides the cast.** A style reference supplies look only - never its character. A science script with no characters gets no people anywhere: not in clips, not on the cover, not in option images.
- **Audio first.** Shot timing comes from measured narration, never from word-count guesses once audio exists.
- **One timing table.** plan.json drives the board captions, the animation prompt, the render length and the edit spans. Mismatches block submission.
- **Labels in code.** Panels come from Codex without text; `assemble_boards.py` prints the SHOT/time/CAMERA/ACTION strips.
- **Inspect everything you generate** before anything depends on it. Record what you looked at; never claim a review you did not do.
- **Paid jobs:** submit each ready clip once, record the real job ID, never resubmit a slow job, keep rejected renders and their IDs.
- **Repairs:** first re-time the edit to the observed cuts (lt-review). Only a real picture defect gets one repair render; a second failure stops for the user.

Project layout

```
projects/<slug>/
  intake.json            script.json          plan.json
  refs/                  style reference (style only)
  audio/                 N*.mp3, narration.json
  assets/                scene masters, prop/macro sheets
  storyboards/panels/    <CLIP>_P<n>.png from Codex
  storyboards/           <clip>_storyboard.png, boards.json
  static/                cover.png, option_*.png, confidence_*.png, check_*.png
  prompts/               codex-*.md (image/audio steps), <clip>.txt (EN2 prompts)
  uploads.json           EN2 upload names + hashes
```

```
renders/                jobs.json, <clip>.mp4, cuts.json, rejected/
review.json             edit.json     output/     logs/
```

Start from `templates/`. Run every pipeline script from the repo root with `--project projects/<slug>`.

Delivery

The ZIP from `package.py` contains the final video, per-clip segments, static images and `roleplay.json` (which image belongs to which option), narration audio and cues, boards, prompts, plan, edit plan, job IDs, review record and a SHA-256 manifest.

Skill: lt-intake

Mandatory Living Textbook intake - ask for and record the script source, the style reference image and the narrator voice before any script building or generation.

Living Textbook - Intake

Ask these three questions and wait for real answers before building the script or generating anything. Reuse answers already given for the same source and version.

1. **Script:** "Which script are we building from - a Google Doc, or a Starling roleplay link?"
2. **Style:** "Please share the style reference image (or a Starling link whose look we should match)."
3. **Narration:** "Which ElevenLabs voice (ID or name) should narrate?" If the script names a narrator voice, confirm it rather than assuming.

There are no character voices in this workflow. If the script contains dialogue, stop and ask whether to convert it into narration or to route that production to the Starling dialogue workflow.

Reading the source

- **Google Doc:** read it through the Google Drive connector (`read_file_content` with comments). The doc is authoritative for words, choices, routes and feedback.
- **Starling link:** read the story through the Starling MCP, or the public story API (`https://starling.en2.ai/api/v1/stories/<slug>`) when MCP is unavailable. Save the JSON. Inspect its actual images (poster, option images) - a text description is not enough.
- **Style reference:** download it to `refs/`, look at it and write a one-paragraph style description (geometry, shading, palette, lighting, background treatment). The reference supplies **style only**: never take its character, face, identity or props into the production.

Record

Save `intake.json` (template in `templates/intake.example.json`):

- `source`: kind (`google_doc` | `starling_link`), URI, local copy path and SHA-256
- `style`: reference path, SHA-256, the written style description, `"use": "style_only"`
- `narrator`: voice ID, name, model (`eleven_v3`)
- `cast_from_script`: the on-screen characters the script actually contains (often none)
- `answers`: the user's actual words or message references for each question
- `confirmed_by: "user"`, `status: "complete"`

Do not invent confirmations. Changed answers invalidate only the stages that depend on them.

Skill: lt-script

Turn a Living Textbook source script into script.json - every narration line split into renderable clips, the choice-to-narration mapping for Predict-Observe-Explain lessons, and the list of static images the roleplay needs.

Living Textbook - Script

Read the complete source (intake). Keep its words exactly; do not rewrite narration unless the user asks. Produce `script.json` (see `templates/script.example.json`).

1. Narration lines

List every spoken line with a stable ID (`N1`, `N2`, ...), the exact text, where it plays (step, branch or option) and what it must show. One audio file per line. Visible text on screen (option labels, captions) is interface text, not narration.

2. Clips

A clip is one EN2 render: a 1x2 or 1x3 storyboard covering one narration line or one part of it. EN2 renders at most 15 s, and every shot carries a 1-2 s buffer, so plan a clip at about 11 s of narration or less. Split longer lines at sentence boundaries (the measured pauses from lt-audio confirm the split). Record for each clip: ID (`C1-stomata`), narration ID, which sentences it covers, and the beats it must show.

3. Choice mapping (Predict-Observe-Explain)

For choice-based lessons record, per option, which narration plays. The default Living Textbook mapping:

- Group every answer and reason by the **idea** behind it (soil, water, sunlight, correct, not sure ...).
- Six cases decide what plays: both right; same wrong idea twice (one merged clip); right answer + wrong reason; wrong answer + right reason; two different wrong ideas; not sure.
- Narration is built from short pieces: `SAY_<option>` ("You said ..."), `LOOK_<idea>` ("Look again: ..." pointing at evidence from the Observe film), `RIGHT`, `WHY` (the "but why ...?" bridge), `POINTS_<answer>`, `NEXT`, `NOT_SURE`, then the common explanation.
- Write the playback order for every answer x reason combination.

If the lesson's player plays one opener only, also record the single-clip fallback (one rendered clip per combination).

4. Static images

List every still the roleplay shows (lt-static-images makes them):

Kind	One per	Example source field
cover	lesson	title / hook
answer	answer option	option text
reason	reason option	reason text
confidence	confidence level	level text
check	check question (and its scenario)	question text
other	any still the script asks for (diagram card, end card)	as written

Give each an ID, the exact option text it illustrates and the idea it must show without words.

5. Inventory

List the on-screen cast **from the script only** (usually none for science lessons), the locations, and the key props/evidence (experiment set-ups, specimens, diagrams). Never add a character because the style reference is a portrait.

Check before handing on

Compare script.json with the source line by line: every narration line, option, feedback line and route is present and identical. Record the comparison in script.json `source_check`.

Skill: lt-audio

Generate Living Textbook narration with ElevenLabs through Codex, download it with the harvester, and measure durations and sentence pauses that drive all shot timing.

Living Textbook - Narration Audio

Audio comes before any visual planning. Codex has the ElevenLabs app connected; Claude cannot call ElevenLabs directly here, and Codex's sandbox cannot download the result, so the step is split in two.

1. Generate (Codex)

Write `prompts/codex-audio.md` from `templates/codex-audio.md`:

- voice ID from intake, model `eleven_v3`
- one file per narration line, path `audio/<N-id>.mp3`, text copied exactly from script.json
- the reply must END with a JSON array: `[{"file": "audio/N1.mp3", "generation_id": "...", "history_id": "...", "voice_id": "...", "model": "eleven_v3"}]`

Run:

```
python pipeline/codex_run.py --project projects/<slug> --prompt prompts/codex-audio.md --name audio
```

Codex will report that it generated the audio but could not save it. That is expected.

2. Harvest and measure

```
python pipeline/harvest_audio.py --project projects/<slug> --name audio
```

The harvester reads the JSON block, finds each generation's signed URL in the Codex log, downloads the MP3 and writes `audio/narration.json` with `seconds` and `pauses` (FFmpeg silence detection, -35 dB, 0.18 s). Download soon after generation; the signed links expire.

3. Listen and accept

Listen to every file completely. Check the words match script.json exactly, pronunciation, pace and that nothing is cut off. Record the verdict per file in narration.json (`accepted: true`, `listened_by`). Regenerate only the failing line.

4. Cues for planning

Sentence boundaries are the midpoints of the measured pauses. A cue inside a sentence (no pause) is an estimate; mark it `estimated` in plan.json. If a narration line is too long for one clip, split it at a measured pause (lt-script).

Keep the original speed. Never stretch or squeeze narration to fit footage; footage is trimmed to the narration (lt-assembly).

Skill: lt-visual-plan

Build the Living Textbook timing table (plan.json) - every narration clip as a 1x2 or 1x3 storyboard with measured required intervals, buffered generated ranges, camera, action and an exact visible inventory, using science-friendly shot types.

Living Textbook - Visual Plan

Input: script.json, audio/narration.json (accepted), intake style description. Output: `plan.json` (see `templates/plan.example.json`). This one table drives the board captions, the animation prompt, the render duration and the edit.

Clip and shot rules (Starling storyboard method)

- Every narration clip is a labelled black-and-white **1x2 or 1x3** storyboard. A single full-frame narration scene is invalid. Choose 2 or 3 shots by meaning, not by length: each shot carries a distinct beat of the line.
- Each shot gets the narration interval it covers (`required`, in the narration file's clock, from measured cues) and a **1-2 s buffer**. Whole-second generated ranges: `generated`, `action`, `hold`. `cut_times` = the generated starts of shots 2..n.
- `render_seconds` = end of the last generated range. EN2 renders 4-15 s; split the clip at a measured pause if it would exceed 15 s.
- `camera`: one move with direction, extent and speed (slow push-in about 5 %, slow tilt up, static close-up, slow pull-back). `action_text`: the start state, the visible motion and the settled state, in one sentence.
- `visible`: the exact inventory for that shot. An empty insert stays empty.

Science shot types

Type	Use for	Notes
Establishing / environment	the place the lesson happens (windowsill, field, lab bench)	from the scene master; no people unless the script has them
Experiment close-up	apparatus doing the thing (bubbles rising, a reading changing)	apparatus layout must match the scene master exactly
Macro insert	surfaces and small structures (leaf underside, stomata)	from a macro prop sheet
Cutaway	inside an object (stem, soil, pipe)	plain background, centred subject
Cross-section / cell view	inside tissue, cells, layers	keep the molecule/particle designs from the sheet
Process close-up	particles moving and changing (CO2 + water -> sugar)	state what enters, what leaves, what appears
Wide return	back to the set after an insert	same scene master, same layout

Keep molecule and particle designs identical across shots (e.g. CO2 = dark sphere between two red spheres; water = blue drop; sugar = white hexagon).

Cast

Characters only if script.json lists them. No people for a cast-free script - plan nature, apparatus and macro shots instead. A style reference never adds a character.

Continuity

One approved scene master per location fixes the camera side, layout (left/right of every object), furniture and lighting direction. Every later shot of that location keeps it. Record the layout in plan.json `locations`.

Hand-off check

Every narration line is covered; required intervals tile each clip's audio interval with no gaps; every generated range is whole seconds; shot counts are 2 or 3; renders are 4-15 s; visible inventories contain only script-supported subjects.

Skill: lt-images

Make Living Textbook visual assets and storyboards with Codex native images - scene masters, prop and macro sheets, black-and-white storyboard panels - then assemble the labelled boards in code and inspect them.

Living Textbook - Assets and Storyboards

All pictures are made by **Codex native image generation** through `pipeline/codex_run.py`. Attach the style reference first on every call. Never generate animation or diagrams in code.

1. Assets (colour)

Write `prompts/codex-assets.md` (template `templates/codex-assets.md`). Make only what the plan's visible inventories need:

- **Scene master** per location: full-frame, no people (unless the script has a cast), fixed layout described left to right, lighting direction.
- **Prop / apparatus sheets** for experiment set-ups.
- **Macro / cutaway / cross-section sheets** for science inserts, with the particle designs fixed.
- **Character sheets** only for script characters (front, three-quarter, side; lips closed).

Prompt rules: the style reference is style only ("do not draw this person"); 16:9; no text, labels or watermarks; one palette and light direction across all assets. Codex copies each PNG from `~/.codex/generated_images/` to the exact project path.

```
python pipeline/codex_run.py --project projects/<slug> --prompt prompts/codex-assets.md --ref
refs/style.jpg --name assets
```

Inspect every asset before continuing: style match, no invented people, layout as described, particle designs, no text. Regenerate only what fails.

2. Storyboard panels (black and white)

Write `prompts/codex-panels.md` (template `templates/codex-panels.md`): one panel per shot, saved as `storyboards/panels/<CLIP>_P<n>.png` where `<CLIP>` is the clip id before the first dash. Attach the style reference, then the assets each panel uses (scene master, sheets).

Panel rules: black, white and grayscale only; 16:9 edge to edge; no text, arrows or motion marks; exact visible inventory from plan.json; the scene master's layout and light; physical actions that make sense (supports, contacts, directions).

3. Assemble the boards (labels in code)

```
python pipeline/assemble_boards.py --project projects/<slug>
```

Stacks the panels top to bottom (Shot 1 on top), forces grayscale and 16:9, and prints a white strip under each picture: `SHOT n g0-g1 s (action a0-a1 s, hold h0-h1 s)`, `CAMERA: ...`, `ACTION: ...` - all from plan.json. Writes `storyboards/boards.json` with hashes and picture boxes.

4. Inspect the boards

For each board check: one distinct beat per panel; layout identical to the scene master; inventory exactly as planned (no extra people or objects); physical logic; captions match plan.json. A failed panel is regenerated as a new version; move the rejected file to `storyboards/panels/rejected/` and rebuild only that board. Record the inspection in plan.json `board_review`.

Skill: lt-static-images

Make the static images a Living Textbook roleplay shows as stills - the cover page and one text-free image per answer, reason, confidence and check option - with Codex in the approved style, and map them in roleplay.json.

Living Textbook - Cover Page and Option Images

The interactive player shows several pictures that are not video: the cover (poster) on the title screen, one picture per answer option, per reason option, per confidence level and per check question. The player overlays the option text, so the pictures carry **no text at all** and must explain each idea visually.

Reference: the Starling monsoon story uses `poster.jpg` (2560 px wide) and `option_A.jpg`, `option_r1.jpg`, `confidence_sure.jpg`, `plateau.jpg` (check) at about 1600 x 900.

Make them

Write `prompts/codex-static.md` from `templates/codex-static.md`, one entry per item in script.json `static`:

File	Content
`static/cover.png`	Hero image of the lesson's subject in the scene master's world, with calm space in the upper third for the title overlay
`static/option_<A..>.png`	The idea of that answer, drawn literally (a wrong idea is drawn as the learner imagines it, not as a correction)
`static/option_r<n>.png`	The idea of that reason
`static/confidence_<level>.png`	A calm visual of that confidence level, matching the lesson world
`static/check_<n>.png`	The check question's scenario

Rules: colour, 16:9, edge to edge; no text, letters, numbers or labeled arrows; no people unless the script has a cast; the approved style reference (style only) plus the scene master and relevant sheets as refs; one palette across the set. Do not reveal the correct answer: right and wrong options get the same visual weight and polish.

```
python pipeline/codex_run.py --project projects/<slug> --prompt prompts/codex-static.md --ref refs/style.jpg --ref assets/<scene_master>.png --name static
```

Inspect

Each image shows its own idea without words; none contains text or extra people; the set looks consistent; the correct option is not visually favoured; the cover has room for the title. Regenerate only failures.

Cover page option in the video

`build_edit.py --cover static/cover.png --cover-seconds 3` opens the stitched video on the cover as a silent still. Leave it out when the player shows the cover itself.

roleplay.json

`package.py` writes `roleplay.json` from plan.json `static`: for every option its ID, exact text, kind and image path, plus the cover. Use it to load the images into the player (Starling story data: `poster`, `answers[].image`, `reasons[].image`, `sures`, `check.image`).

Skill: lt-animation

Write six-section EN2 reference prompts from the inspected Living Textbook storyboards, upload references with hash checks, submit all clips once on Ultra, and collect the renders.

Living Textbook - Animation (EN2, Ultra)

Inputs: inspected boards (boards.json), inspected assets, plan.json. Read the live EN2 reference guide (`video_guide mode=reference`) and `gpu_timing` once per session.

1. Choose references

Order: visible characters (only if the script has them) -> matching environment / scene master -> props, macro and cutaway sheets actually visible in the clip -> **storyboard last**. Attach only what the clip's shots show. An environment-free science clip (e.g. cell cross-section) may be just its sheet + the board.

2. Write the prompt (one per clip, `prompts/<clip>.txt`)

Use the six sections in order (`templates/en2-prompt-template.txt`):

1. `subject_definitions` - `<Subject N>` for each environment / sheet with the features to keep, from `<Picture N>`; the board is its own `<Picture N>` mapping top-to-bottom pictures to Shot 1..3.
2. `summary` - `[reference generation]`, duration, 3D/2.5D/2D style from the assets, the shots, the hard-cut times.
3. `retention_analysis` - one line per Subject / Picture with `fully_preserved` or `partially_preserved` and the shots it appears in.
4. `detailed_description` - a style sentence, then: "Render each illustrated scene individually as an edge-to-edge, full-frame colour shot. At the specified time, switch directly to the next fully formed shot using a hard cut. Treat the storyboard layout and caption strips exclusively as production guidance." Then per shot: generated range, the **actual inspected panel** described in detail (composition, positions, objects, background, light), `ACTION a0-a1 s` (start state, motion, settled state), `CAMERA` (move and extent), `HOLD h0-h1 s`. Shots 2+ begin `At 00:0X.000, hard cut to ...`. For a cast-free script add "No people appear."
5. `overall_soundscape: Silent visual sequence; approved original narration is added during assembly.`
6. `non_diegetic_music: N/A`

Affirmative instructions only. Timings must equal plan.json and the printed board labels.

3. Upload (hash-checked)

```
 cd projects/<slug>
node ../../pipeline/en2_upload.mjs assets/<...>.png storyboards/<clip>_storyboard.png >
uploads.json
```

The uploader calls the EN2 MCP `video_upload` with the file bytes (the MCP URL is read from `EN2_MCP_URL` or `~/.codex/config.toml` and never printed). EN2 names uploads `en2ref_<first 16 hex of SHA-256>`; confirm every name matches the local hash.

4. Submit once, Ultra

`video_generate` with `refs` in the recorded order, `duration = render_seconds`, **`quality = true`**, `folder = "Living Textbook/<Project>/<Batch>"`. Submit every ready clip back to back so they share warm GPUs. Record each response in `renders/jobs.json` (job ID, refs, duration, seed, folder). Never resubmit a slow job; a cold start adds minutes. Tell the user `studio_queue`'s estimate in plain words.

5. Collect

```
node ../../pipeline/en2_collect.mjs renders/jobs.json
```

Polls `video_status` every 30 s and saves `renders/<clip>.mp4`. Hand each finished clip to lt-review straight away.

Skill: lt-review

Review every Living Textbook EN2 render - detect real cut times, re-time before repairing, inspect each shot against its storyboard panel, record a verdict, and allow one repair render per clip.

Living Textbook - Review and Repair

1. Technical pass

```
python pipeline/review_cuts.py --project projects/<slug>
```

For each `renders/<clip>.mp4`: FFmpeg scene detection finds hard cuts (score >= 0.3) and soft transitions, compares them with plan.json `cut_times`, flags drift over 0.5 s, and writes a 2-fps timestamped frame sheet `renders/<clip>.review.png`. Results go to `renders/cuts.json`.

2. Visual pass (a person / the assistant looking)

Open the frame sheet and play the clip. Check every shot against its panel:

- correct shot order and hard cuts (no dissolves, mixed panels, borders, captions or storyboard layout in the picture)
- exact inventory: no extra, duplicated or missing objects; **no people unless the script has them**
- layout identical to the scene master (left/right positions, camera side, lighting)
- the labelled action happens (start state, motion, settled state) and the camera move
- style and colours match the assets; particle designs consistent

Record in `review.json` per clip: `visual_status` (`verified`, `verified_with_retime`, `needs_fix`, `unverified`), per-shot notes, observed cuts, and what was actually inspected. A frame sheet alone is a diagnosis aid; say so if full playback was not done.

3. Cut drift: re-time first

EN2 often moves cuts by a few tenths of a second and sometimes by seconds. Before calling a repair, run `build_edit.py`: it rebuilds the spans from the observed cuts and carries any shortfall of one shot into the next shot. If it succeeds, the clip is `verified_with_retime` (the cut lands slightly before a sentence boundary, which is fine). Only if the footage cannot cover the narration does the clip become `needs_fix`.

4. Repair (once per clip)

For a real picture defect: keep the original and its job ID (`renders/rejected/`), fix the cause at its stage (panel, asset or prompt), inspect the fresh board, write a simpler, clearer prompt for the same beat, and submit one Ultra replacement under the same logical clip ID. Review it completely. If it fails again, stop and show the user both versions.

Minor issues that do not change the meaning (a faint detail, a molecule stopping at a pore) are noted, not repaired.

Skill: lt-assembly

Stitch verified Living Textbook clips under the original narration with FFmpeg - spans from observed cuts, optional cover page opener, per-clip segments, final video - and pack the delivery ZIP with roleplay.json.

Living Textbook - Edit, Stitch and Package

1. Edit plan

```
python pipeline/build_edit.py --project projects/<slug> [--cover static/cover.png --cover-seconds 3]
```

Builds `edit.json` from plan.json, `renders/cuts.json`, `review.json` and `audio/narration.json`. Only `verified*` clips are included (`--include-unverified` for previews only). Each shot's span starts at its observed cut and is as long as its narration interval; shortfalls carry into the next shot; the narration is never stretched.

2. Stitch

```
python pipeline/assemble_edit.py --project projects/<slug>
```

For every segment: trim the spans, join with hard cuts, scale/pad to the project size and fps, discard EN2's sound and lay the narration interval underneath. Writes `output/segments/NN_<clip>.mp4` (one per clip, useful when the player plays clips separately - e.g. choice openers) and `output/final.mp4` (all segments in order, the cover first if requested).

3. Check the result

Play the final video completely: every cut, every narration word, no gaps or doubled audio, picture matches the narration it sits under, cover (if any) correct. Note the check in review.json `final_check`.

4. Package

```
python pipeline/package.py --project projects/<slug>
```

Creates `<slug>-living-textbook-<date>.zip` with: output video and segments, static images and `roleplay.json` (cover + every option with its text, kind and image), narration MP3s and narration.json, boards and panels, assets, prompts, plan.json, edit.json, job IDs, cuts.json, review.json, intake.json, and `MANIFEST.json` (SHA-256 per file). Rejected media stays out of the ZIP but remains in the project.